Take-Home Assignment: PermitFlow

Sep 30, 2026

Overview

Build **PermitFlow**, a small multi-tenant web app where organizers request event permits and several agencies review them, in **3 days** from the time you receive this brief.

We work AI-first at LetUsCode. You are expected to use Claude Code, Cursor or similar tools heavily. We are not testing whether you can type code fast. We are testing whether you understand what you are building, direct the AI well, and can explain every part of the result.

Rules

- AI coding tools are allowed and encouraged. Use whichever you like.
- Keep your AI chat and prompt history. You will share it at the end (export, screenshots or copied logs are all fine).
- Commit to Git early and often, with meaningful commit messages. We will read the history.
- Do not start from a paid SaaS boilerplate or someone else's finished repo. Official starters (`create-next-app`, FastAPI templates) are fine.
- If a requirement is unclear, make a sensible decision and write it down in `DECISIONS.md`. You may also ask us one round of questions on day 1.
- A working core is worth more than a half-built everything. Prioritise the **Must** items.

After submission we will run a 60–90 minute live session where you extend and change the app with your AI tools while sharing your screen.

Product brief

Cities receive permit requests for public events such as marathons, concerts and street fairs. Today these arrive by email, and each agency (Police, Fire, Health) reviews them separately. PermitFlow puts one request in front of all the agencies that need to sign off, and tracks it until it is approved or rejected.

Each **city is a tenant**. Data from one city must never be visible to another city.

Roles

Role	Belongs to	What they do
Organizer	A tenant (city)	Submits permit applications, uploads documents, answers reviewer questions, tracks status
Agency Reviewer	One agency within a tenant	Sees applications assigned to their agency, approves, rejects or requests changes, with comments
City Admin	A tenant	Manages agencies and users, decides which agencies review which event types, sees everything in their city

Application lifecycle

Draft → Submitted → Under Review → Approved, Rejected, or Changes Requested. Changes Requested goes back to the organizer, who edits and resubmits. An application is Approved only when **every** required agency has approved it. It is Rejected if **any** agency rejects it.

Functional requirements

Must (core — build these first)

1. **Auth and tenancy.** Email + password login (JWT is fine). Every user belongs to exactly one tenant and has one role. Seed two tenants (e.g. "Springfield" and "Riverton") with demo users for every role.
2. **Tenant isolation.** Enforce it in the database with PostgreSQL Row-Level Security, not only in API filters. A Riverton user must get nothing from Springfield, even by guessing IDs.
3. **Organizer: create application.** Fields: event name, event type (Marathon, Concert, Street Fair, Other), date(s), location, expected attendance, description. Save as draft, then submit.
4. **Routing.** On submit, create one review task per required agency. City Admin configures which agencies are required per event type. Rule: attendance over 5,000 always adds Fire.

5. **Reviewer: review queue.** List of applications assigned to my agency, filterable by status. Detail view with Approve, Reject, Request Changes, each with a mandatory comment.
6. **Status logic.** Overall status follows the lifecycle above, computed from the agency decisions. Invalid transitions are refused by the API (e.g. approving an application still in Draft).
7. **Organizer: tracking.** Organizer sees overall status plus each agency's decision and comments, and can edit and resubmit after Changes Requested. Resubmission resets only the agencies that requested changes.
8. **Audit trail.** Every status change and decision is recorded: who, what, when. Shown as a timeline on the application page.

Should (if time allows)

- Document upload (site map, insurance certificate) stored in S3 or local disk behind a storage interface.
- City Admin dashboard: counts by status, average days to decision.
- Basic tests for status logic and tenant isolation.

Could (nice to have)

- Email or in-app notifications on status change.
- Docker Compose for one-command local setup.

Tech stack and constraints

Use our production stack so the result is something we could actually extend.

Layer	Required
Frontend	Next.js (App Router), TypeScript
Backend	FastAPI, Python 3.11+
Data	PostgreSQL with RLS, SQLAlchemy 2.0, Alembic migrations
Validation	Pydantic v2 schemas on every API boundary
Styling	Your choice (Tailwind, shadcn/ui, etc.)

- Backend and frontend in one repo, two folders (`/api`, `/web`).
- Status transitions live in one place in the backend, not scattered across routes or the UI.
- No secrets in the repo. Use `.env.example`.
- It must run locally from the README steps on a clean Mac.

Deliverables

Submit a GitHub repo link by the end of day 3, containing:

- [ ] Working code for all **Must** items, with seed data and demo logins
- [ ] `README.md` — setup and run steps, demo credentials
- [ ] `ARCHITECTURE.md` — **written by you, not generated**: folder structure, data model (tables and relationships), how RLS is wired, how a request flows from the UI to the database, where the status logic lives and why
- [ ] `DECISIONS.md` — assumptions you made and trade-offs you chose
- [ ] `AI_LOG.md` — your main prompts and AI sessions (exports or pasted), plus 3–5 moments where the AI got it wrong and how you noticed and fixed it
- [ ] A 5-minute screen recording walking through the three roles end to end

In the live session we will ask you to change the app and to explain your code without the AI's help, so make sure you understand everything you commit.